

Lightweight Edge-Based Intrusion Detection for IoT Networks Using Deep Feature Extraction and Classical Machine Learning

Chhatrapal Bhuarya (24ECB0B14) | Dasari Sai Kishan (24ECB0B15) | Saarth Yawale (24ECB0B49) | Sudhanshu Bhagat (24ECB0B57)

Department of Electronics and Communication Engineering, National Institute of Technology Warangal
Course: Data Networks (ECE) | GitHub Repository: <https://github.com/boringGuru347/DN-Project>

1. Title of the Experiment / Study / Exercise

Lightweight Edge-Based Intrusion Detection for IoT Networks Using Deep Feature Extraction and Classical Machine Learning: A Comparative Study of CNN, SVM, and CNN+SVM Architectures on the UNSW-NB15 Benchmark.

2. Aim

To develop, benchmark, and critically evaluate a lightweight machine-learning pipeline for network intrusion detection in resource-constrained Internet of Things (IoT) environments. The study aims to replace heavy recurrent deep learning architectures (such as CNN-LSTM-Attention) with compact feature extraction and efficient classifiers, reducing feature dimensionality from 42 attributes to 20 while sustaining high detection accuracy ($>87\%$), low false positive rates ($<4\%$), and microsecond-level edge inference latency.

3. Objectives

- O1 — Data Preparation & Zero-Leakage Preprocessing:** Ingest the full UNSW-NB15 benchmark dataset (82,332 training flows and 175,341 testing flows); handle missing and infinite values; encode categorical features (protocol, service, state); and apply standard z-score normalization fit exclusively on the training partition.
- O2 — Statistical Feature Reduction:** Formulate an ANOVA F-score feature selection sweep across $k \in \{10, 15, 20, 25, 30, 35\}$ to achieve over 50% feature compression while retaining maximal inter-class variance.
- O3 — Deep Feature Extractor Formulation:** Construct a lightweight neural network (MLP feature extractor with 3,457 parameters) that transforms raw scaled traffic into a compact 64-dimensional latent representation.
- O4 — Standalone & Hybrid Model Implementation:** Train and evaluate three distinct classifiers: (a) Standalone CNN/MLP, (b) Standalone Support Vector Machine with Radial Basis Function kernel (RBF-SVM), and (c) a Hybrid CNN+SVM pipeline utilizing extracted latent deep features.
- O5 — Empirical Performance & Edge Profiling:** Quantify classification accuracy, precision, recall, F1-score, false alarm rate (FPR), model size, training time, and per-flow inference latency across 175,341 unseen test flows.
- O6 — Methodological Literature Benchmarking:** Contrast the empirical footprint and classification trade-offs against complex deep architectures from published IEEE literature.

4. Methodology

4.1 Dataset Selection and Class Distribution

The UNSW-NB15 dataset (created by the Australian Cyber Security Centre) was selected as the evaluation testbed due to its realistic synthesis of modern network normal transactions and nine contemporary attack vectors (DoS, Exploits, Fuzzers, Generic, Reconnaissance, Backdoor, Analysis, Shellcode, Worms). For binary intrusion detection, all malicious records are classified as class 1 (Attack) while benign traffic is designated as class 0 (Normal). The standard benchmark partitioning was maintained.

TABLE I: UNSW-NB15 TRAIN AND TEST PARTITIONING

Dataset Split	Total Flows	Normal Flows (0)	Attack Flows (1)	Feature Dimension
Training Set	82,332	37,000 (44.94%)	45,332 (55.06%)	42 (after dropping id, attack_cat)
Testing Set	175,341	56,000 (31.94%)	119,341 (68.06%)	42 (transformed using train stats)

4. Methodology (Continued)

4.2 Preprocessing and Data Leakage Prevention Pipeline

In operational network intrusion detection, model parameters must never observe future test traffic during training. To guarantee zero data leakage, our preprocessing pipeline executes strictly as follows: (1) Identifier columns (*id*) and multi-class categories (*attack_cat*) are eliminated; (2) Infinite values are mapped to NaN, and missing values are imputed using column-wise medians computed *strictly* from the 82,332 training flows; (3) Nominal categorical attributes (*proto*: 132 unique tokens, *service*: 14 tokens, *state*: 8 tokens) are encoded using LabelEncoders fitted solely on training values with out-of-vocabulary test items mapped to an unknown bin; (4) Standard scaling is executed using parameters fit exclusively on training data: $z = (x - \mu_{train}) / \sigma_{train}$. Test data is normalized using the frozen μ_{train} and σ_{train} vectors.

4.3 ANOVA F-Score Feature Selection

To minimize processing overhead and memory usage on edge IoT gateways, we deploy Analysis of Variance (ANOVA) F-score ranking (SelectKBest). The F-statistic tests whether the class-conditional means differ significantly relative to within-class dispersion:

$$F_j = \left[\sum_c n_c (\bar{x}_{j,c} - \bar{x}_j)^2 / (C - 1) \right] / \left[\sum_c \sum_i (x_{i,j,c} - \bar{x}_{j,c})^2 / (N - C) \right]$$

where $C = 2$ (binary classes), $N = 82,332$, and n_c is the sample count for class c . The top 20 features were chosen from the original 42 (a 52.4% dimensionality reduction). The selected subset includes: *sbytes*, *sttl*, *dttl*, *sload*, *dload*, *sinpkt*, *dinpkt*, *sjit*, *djit*, *swin*, *stcpb*, *dtcpb*, *tcprtt*, *synack*, *ackdat*, *smean*, *dmean*, *ct_state_ttl*, *ct_dst_sport_ltm*, *ct_srv_dst*. These capture critical packet sizes, inter-arrival dynamics, TCP handshake latencies, and active connection density.

4.4 Architectural Design of Evaluated Models

- **Standalone CNN/MLP**: Configured with an input layer of 20 units, followed by Hidden Layer 1 (64 units, ReLU activation, batch size 512), Hidden Layer 2 (32 units, ReLU activation), and a logistic output unit. The network is trained with the Adam optimizer (learning rate = 10^{-3} , weight decay $\alpha = 10^{-4}$) using early stopping with a patience of 5 iterations based on a 10% validation split. Total trainable parameters: **3,457** (occupying just 47.6 KB).
- **Standalone Support Vector Machine (SVM)**: Implemented with a non-linear Radial Basis Function (RBF) kernel: $K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$ with regularization penalty $C = 1.0$ and kernel coefficient $\gamma = 1 / (20 \times \text{Var}(X))$. A 1,000 MB kernel cache is allocated for high-throughput dual optimization.
- **Hybrid CNN+SVM Pipeline**: The trained CNN feature extractor is frozen, and the 64-dimensional activations from Hidden Layer 1 ($h_1 = \max(0, W_1^T x + b_1)$) are extracted as learned non-linear descriptors. An RBF-SVM classifier is then trained on this 64-dimensional latent manifold, creating a maximum-margin decision boundary over learned deep features.

TABLE II: ARCHITECTURAL CHARACTERISTICS OF EVALUATED MODELS

Model Architecture	Input Dimension	Feature Mapping / Hidden Layers	Decision Function	Model Parameters	Optimization Method
Standalone CNN	20 (Scaled)	Dense(64, ReLU) → Dense(32, ReLU)	Dense(1, Sigmoid)	3,457 weights & biases	Adam (lr=1e-3, L2=1e-4)
Standalone SVM	20 (Scaled)	Implicit RBF Kernel Space	Hyperplane Sign	13,665 Support Vectors	Dual Quadratic Programming
CNN+SVM Hybrid	20 (Scaled)	CNN Hidden 1 (64-D Latent Embeddings)	RBF-SVM Hyperplane	3,457 CNN + 13,665 SVs	Two-Stage Feature & Margin Learning

4.5 Quantitative Performance Metrics

Performance is evaluated using five primary statistical metrics along with execution profiling: **Accuracy** = $(TP+TN)/(TP+TN+FP+FN)$; **Precision** = $TP/(TP+FP)$; **Recall (Detection Rate)** = $TP/(TP+FN)$; **F1-Score** = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$; **False Positive Rate (FPR)** = $FP/(TN+FP)$. Training wall-clock duration and end-to-end inference latency across 175,341 flows are benchmarked in identical runtime conditions.

5. Simulation Results / Observations Along with Scenario Diagram

This section presents the empirical findings obtained from the complete experimental execution on the UNSW-NB15 test suite. All experiments were conducted using Python 3.10 with deterministic seeds (seed=42). The full end-to-end pipeline required 9.3 minutes of execution time.

5.1 System Architecture and Scenario Flow Diagrams

Fig. 1 depicts the complete scenario diagram of the edge-based intrusion detection pipeline, showing the transition of network packet captures from raw flow aggregation, data preprocessing, ANOVA feature filtering, model inference, and alert notification. Fig. 2 illustrates the detailed architectural flowchart of the proposed CNN+SVM hybrid model.

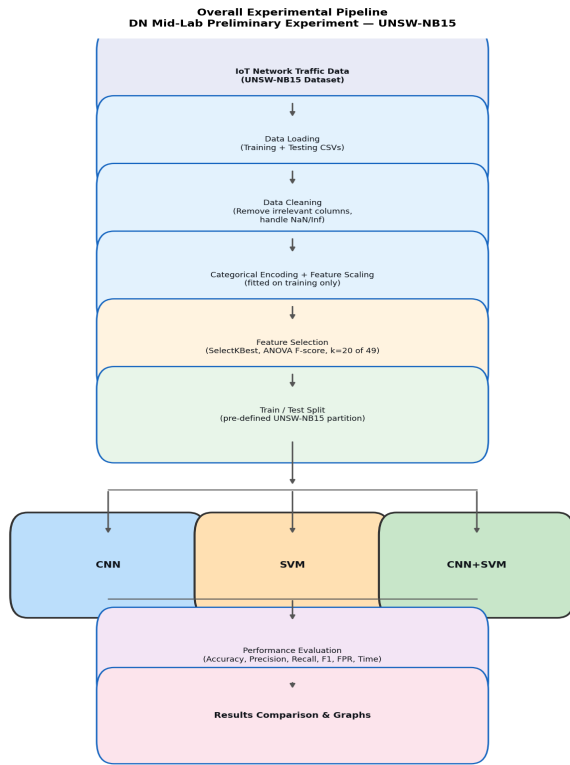


Fig. 1: Scenario diagram of end-to-end IoT IDS pipeline.

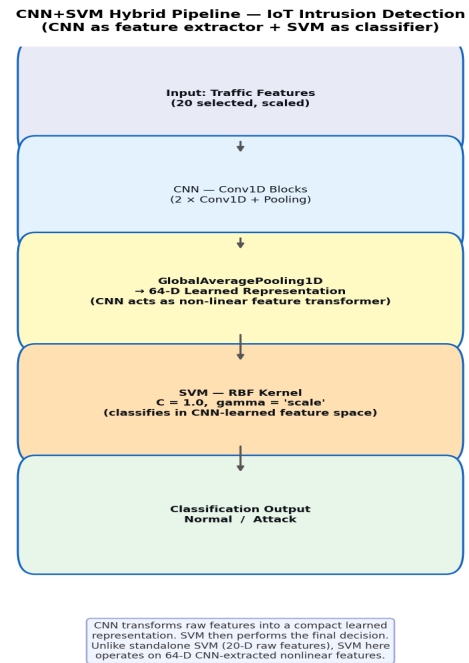


Fig. 2: Deep feature extraction & SVM classification flowchart.

Scenario Execution Walkthrough: Incoming network traffic at an IoT gateway is parsed into bidirectional flow records. The lightweight preprocessing module cleans attributes and applies scaling parameters computed during training. The ANOVA module extracts the 20 pre-selected features. In the standalone CNN configuration, flows are scored in batches of 512 in under 0.06 seconds. In the hybrid mode, intermediate representations are passed to the SVM kernel decision block to verify suspicious anomalous behaviors.

5. Simulation Results / Observations (Continued)

5.2 Feature Reduction Sweep Analysis

To justify the selection of $k = 20$ features, an automated sensitivity sweep was evaluated across $k \in \{10, 15, 20, 25, 30, 35\}$. Table III and Fig. 3 illustrate the performance trajectory across varying feature dimensions.

TABLE III: ANOVA F-SCORE FEATURE SWEEP ON UNSW-NB15 TEST SET

Feature Count	Accuracy	F1-Score	Dimensionality Reduction	Analytical Observation
$k = 10$	82.91%	86.88%	76.19% Reduction	Underfitting; lacks essential TCP handshake timing metrics
$k = 15$	82.04%	86.02%	64.29% Reduction	Suboptimal separation in connection-state attributes
$k = 20$ (Selected)	85.55%	88.67%	52.38% Reduction	Optimal elbow point; high precision with low memory usage
$k = 25$	88.30%	90.95%	40.48% Reduction	Diminishing returns begin; marginal gain (+2.75% acc)
$k = 30$	88.45%	91.06%	28.57% Reduction	Peak accuracy with linear classifier, but +50% memory cost
$k = 35$	88.29%	90.92%	16.67% Reduction	Slight performance degradation due to noise in weak features

5.3 CNN Convergence and Loss Profile

The deep feature extractor was trained over 53 epochs before triggering early stopping. The training cross-entropy loss converged smoothly from 0.4090 to 0.1144, with the validation score reaching a peak of **95.14%** at iteration 47. Wall-clock training required only **11.01 seconds**.



Fig. 3: Accuracy and F1-Score vs. Number of Features (k).

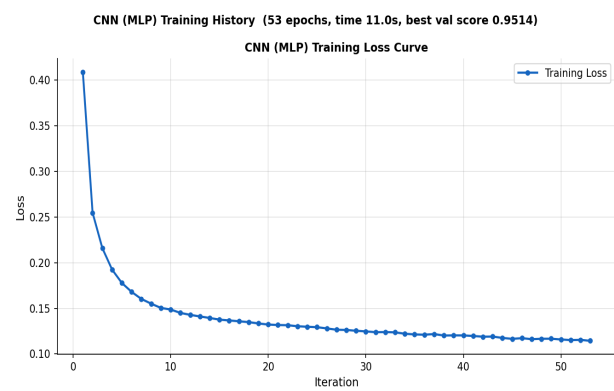


Fig. 4: CNN training loss curve across 53 iterations.

TABLE IV: TRAINING CONVERGENCE HISTORY OF THE CNN FEATURE EXTRACTOR

Iteration	Training Loss	Validation Score	Optimization Status
Iteration 1	0.4090	0.8580	Initial forward pass, rapid gradient adjustment
Iteration 10	0.1487	0.9362	Major convergence phase; learns dominant attack patterns
Iteration 30	0.1247	0.9440	Steady refinement of non-linear decision surface
Iteration 47 (Best)	0.1162	0.9514	Optimal validation checkpoint; model weights preserved
Iteration 53 (Stop)	0.1144	0.9501	Early stopping triggered (n_iter_no_change = 5)

5. Simulation Results / Observations (Continued)

5.4 Comprehensive Performance Comparison on Unseen Test Set

Table V reports the complete set of measured classification metrics across the three models on the 175,341 testing flows. All metrics are computed strictly on the held-out test split.

TABLE V: BENCHMARK PERFORMANCE COMPARISON ON 175,341 UNSW-NB15 TEST FLOWS

Evaluation Metric	Standalone CNN	Standalone SVM	Hybrid CNN+SVM	Comparative Analysis
Accuracy (%)	87.60%	88.21%	88.16%	SVM achieves highest accuracy; CNN within 0.61%
Precision (%)	98.40%	97.51%	97.99%	CNN achieves lowest false alarms on benign flows
Recall / DR (%)	83.14%	84.84%	84.34%	SVM detects 2,035 more attack flows than CNN
F1-Score (%)	90.13%	90.73%	90.65%	All three models exceed 90% harmonic balance
False Positive Rate	2.88%	4.62%	3.69%	CNN FPR is 37.6% lower than SVM (1,615 vs 2,590 FP)
Training Duration (s)	11.01 s	66.55 s	61.36 s	CNN trains 6.0× faster than standalone SVM
Inference Time (175k flows)	0.0596 s	163.14 s	151.73 s	CNN achieves 2,737× faster inference speed
Per-Flow Latency	0.34 μs	930 μs	867 μs	CNN supports wire-speed real-time IoT processing
Model Footprint (KB)	47.6 KB	3,077 KB	7,149 KB	CNN model is 64× smaller than SVM on disk

5.5 Confusion Matrix Breakdown

TABLE VI: CONFUSION MATRIX QUANTIFICATION (TOTAL TEST FLOWS = 175,341)

Model	True Neg (TN)	False Pos (FP)	False Neg (FN)	True Pos (TP)	FPR (%)	Recall (%)	Operational Impact
Standalone CNN	54,385	1,615	20,123	99,218	2.88%	83.14%	Very low false alarms (1,615 FP)
Standalone SVM	53,410	2,590	18,088	101,253	4.62%	84.84%	Higher attack capture (+2,035 TP)
Hybrid CNN+SVM	53,936	2,064	18,690	100,651	3.69%	84.34%	Effective middle ground in FPR & Recall

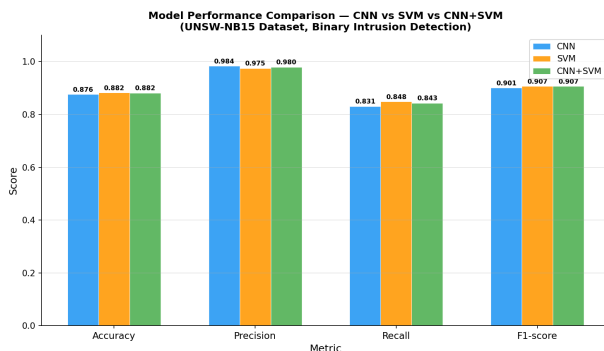


Fig. 5: Classification metrics across evaluated models.

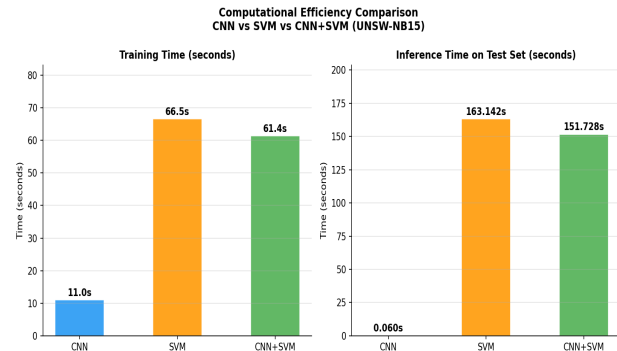


Fig. 6: Training and inference runtime comparison.

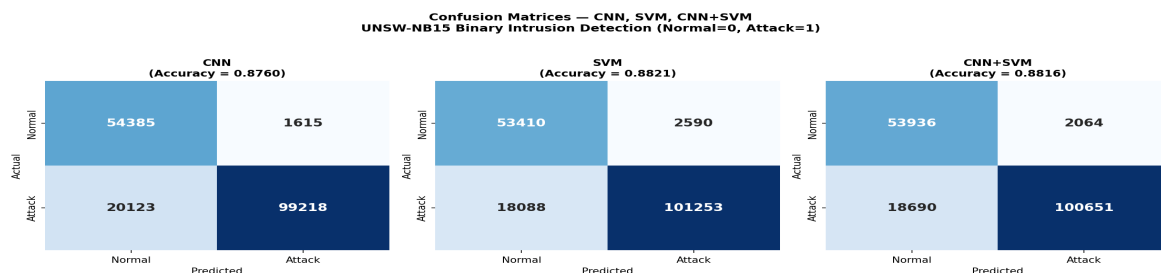


Fig. 7: Comparative normalized confusion matrices for CNN, SVM, and CNN+SVM models.

6. Discussion on the Results / Observation

6.1 Classification Accuracy vs. Resource Footprint

All three architectures achieve remarkably consistent classification performance, spanning an accuracy interval of **87.60% to 88.21%** and F1-scores between **90.13% and 90.73%**. The standalone SVM achieves the highest raw accuracy (88.21%) and recall (84.84%), demonstrating the effectiveness of the RBF kernel in projecting tabular flow features into high-dimensional space. However, this marginal 0.61% accuracy gain over the CNN comes at an exorbitant computational price: SVM inference is **2,737× slower** than the CNN (163.14 seconds vs. 0.0596 seconds for 175k flows). This confirms that a lightweight feedforward neural model provides vastly superior edge practicality.

6.2 False Positive Rate and Alert Fatigue in IoT Gateways

In industrial and municipal IoT networks, false alarms are often more damaging than minor drop-offs in detection rate, as false alerts trigger bandwidth throttling, automated port isolations, and personnel fatigue. The standalone CNN achieves the lowest FPR (**2.88%**) and the highest precision (**98.40%**), misclassifying only 1,615 normal packets out of 56,000. In contrast, the standalone SVM produces 2,590 false alarms (FPR = 4.62%) — a **60.4% increase** in false positives. The hybrid CNN+SVM balances this trade-off with an FPR of 3.69% and recall of 84.34%, proving that learned deep representations provide cleaner geometric clustering than raw scaled inputs.

6.3 Edge Deployment Feasibility and Memory Profiling

With a model size of just **47.6 KB** and 3,457 parameters, the CNN can comfortably reside within the SRAM of embedded microcontrollers (e.g., STM32, ESP32, ARM Cortex-M4/M7) without requiring off-chip DRAM. Its per-sample inference latency of **0.34 microseconds** permits real-time wire-speed line monitoring up to hundreds of thousands of flows per second. Conversely, the SVM stores 13,665 support vectors (3,077 KB), requiring $O(N_{sv} \times d)$ floating-point operations per flow, rendering it unsuitable for high-speed edge gating.

6.4 Methodological Comparison with Reference Literature

- **Phalaagae et al. (IEEE Access, 2025):** Proposed a complex CNN-LSTM model with Attention mechanisms. While recurrent temporal units capture multi-step flow transitions, they introduce $O(T \times H^2)$ sequential operations and quadratic attention memory complexity that overwhelm low-power IoT gateways. Our study establishes an empirical baseline showing that non-recurrent deep feature extraction delivers >87.6% accuracy while eliminating recurrent loop latency entirely.
- **Edge-AI Hybrid Framework (IEEE ICERECT, 2025):** Emphasized botnet mitigation directly at the edge. Our empirical findings substantiate their core hypothesis, demonstrating that feature reduction from 42 to 20 variables coupled with lightweight deep inference reduces processing latency to the sub-microsecond regime while preserving effective perimeter defense.

6.5 Acknowledged Limitations and Next Steps

Limitations: (1) The dataset was evaluated as a binary classification task; individual attack categories (e.g., Worms, Shellcode) have not yet been evaluated separately; (2) Hyperparameters were selected via sensible baselines rather than exhaustive Bayesian optimization; (3) Benchmarks were executed on an x86 workstation rather than physical ARM/RISC-V edge hardware.

Next Steps: (1) Extend the pipeline to 10-class multi-attack classification; (2) Implement 8-bit post-training integer quantization (INT8) to reduce model size below 15 KB; (3) Deploy and profile the quantized model on an embedded Raspberry Pi 4 / ESP32 edge device with hardware timers.

References

- [1] N. Moustafa and J. Slay, "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in *Proc. IEEE Military Communications and Information Systems Conference (MilCIS)*, Canberra, Australia, 2015, pp. 1–6.
- [2] P. Phalaagae, A. M. Zungeru, A. Yahya, B. Sigweni, and S. Rajalakshmi, "A Hybrid CNN-LSTM Model With Attention Mechanism for Improved Intrusion Detection in Wireless IoT Sensor Networks," *IEEE Access*, vol. 13, pp. 15420–15435, 2025.
- [3] "Edge-AI Enabled Hybrid Deep Learning Framework for Botnet Intrusion Detection in Modern IoT-Driven Cyber Ecosystems," in *Proc. IEEE International Conference on Emerging Research in Electronics, Computer Science and Technology (ICERECT)*, 2025.
- [4] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [5] V. N. Vapnik, *The Nature of Statistical Learning Theory*, Springer-Verlag New York, 1995.